

TRƯỜNG ĐẠI HỌC HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
NHẬP MÔN XỬ LÝ ẢNH SỐ
NGÀNH: CÔNG NGHỆ THÔNG TIN

Tên Đồ án:

**PHÁT HIỆN VÀ ĐẾM SỐ LƯỢNG ĐỐI TƯỢNG
TRONG VIDEO (NGƯỜI, XE, VẬT THỂ)**

SVTH:

Nguyễn Huỳnh Anh Tú - 2374802010536

Lê Hoàng Hiệp - 2374802010139

Nguyễn Xuân Công Sơn - 2374802013440

Nguyễn Phước Thành – 2374802013578

GVHD: TS.Đỗ Hữu Quân

Lớp: 243_71ITAI40803_01 Nhóm: 20

TP. Hồ Chí Minh – năm 2025

LỜI CẢM ƠN

Viết một báo cáo đồ án môn học là một trong những việc khó nhất mà chúng em phải hoàn thành trong quá trình học một môn học. Trong quá trình thực hiện đề tài chúng em đã gặp rất nhiều khó khăn và bế ngõ. Nếu không có những sự giúp đỡ và lời động viên chân thành của nhiều người có lẽ chúng em khó có thể hoàn thành tốt tiểu luận này. Đầu tiên chúng em xin gửi lời biết ơn chân thành đến thầy Đỗ Hữu Quân, người trực tiếp hướng dẫn chúng em hoàn thành tiểu luận này. Những ý kiến đóng góp của thầy là vô cùng hữu ích, nó giúp chúng em nhận ra các khuyết điểm của đồ án. Cảm ơn thầy và các bạn trường Đại học Văn Lang là những người đã cùng nhóm em sát cánh và trải nghiệm để hoàn thành đồ án môn học.

Nhóm thực hiện báo cáo

MỤC LỤC

CHƯƠNG 1 : CƠ SỞ LÝ THUYẾT.....	1
1.1. Giới thiệu đề tài	1
1.2. Tổng quan về bài toán phát hiện đối tượng (Object Dectection).....	1
1.3. Giới thiệu về mô hình YOLO	2
1.4. Giới thiệu về thư viện ByteTrack.....	7
1.5. Giới thiệu về thư viện Ultralytics	8
CHƯƠNG 2: PHÂN TÍCH VÀ CÀI ĐẶT.....	9
2.1. Công cụ sử dụng	9
2.2. Phân tích bài toán	9
2.3. Kiến trúc hệ thống tổng quát.....	10
2.4. Mô tả mô hình YOLOv8 sử dụng.	10
2.5. Mã nguồn chính	11
2.6. Kết quả đầu ra	13
2.7. Sơ đồ hệ thống.....	14
2.8. Nhận xét	15
Ưu điểm:	15
Nhược điểm:	15
CHƯƠNG 3 : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	16
3.1. Hướng phát triển	16
3.2. Kết luận.....	16
TÀI LIỆU THAM KHẢO	18

CHƯƠNG 1 : CƠ SỞ LÝ THUYẾT

1.1. Giới thiệu đề tài

Giới thiệu bài toán: Phát hiện và đếm đối tượng (người, xe, vật thể) trong video là một trong những ứng dụng quan trọng của trí tuệ nhân tạo (AI) và thị giác máy tính (Computer Vision). Bài toán này yêu cầu hệ thống vừa phải xác định chính xác vị trí của các đối tượng xuất hiện trong từng khung hình, vừa phải theo dõi và đếm số lượng đối tượng di chuyển qua một vùng hoặc thời điểm nhất định..

Việc triển khai hệ thống phát hiện và đếm đối tượng có nhiều ứng dụng thực tiễn:

- Giám sát giao thông: Giúp đếm số lượng phương tiện, phát hiện ùn tắc, hỗ trợ điều khiển đèn tín hiệu thông minh, và giám sát vi phạm (vượt đèn đỏ, đi sai làn...).
- Đếm người ra vào: Ứng dụng trong các tòa nhà, siêu thị, sự kiện đông người để kiểm soát lưu lượng khách, tối ưu nhân sự hoặc đảm bảo an toàn.
- Theo dõi an ninh tự động: Giúp phát hiện các hoạt động bất thường, theo dõi đối tượng khả nghi, hoặc hỗ trợ điều tra sau sự cố.
- Ứng dụng trong sản xuất: Đếm số sản phẩm trên dây chuyền, phát hiện sản phẩm lỗi, hỗ trợ quản lý kho bãi.

Mục tiêu đề tài:

- Tìm hiểu thuật toán phát hiện đối tượng (Object Detection): Nắm vững nguyên lý, ưu nhược điểm, và các phương pháp phổ biến.
- Áp dụng mô hình YOLOv8: Sử dụng một trong những mô hình phát hiện đối tượng mới và mạnh mẽ nhất hiện nay.
- Thực hiện đếm chính xác số lượng vật thể qua vùng xác định: Kết hợp phát hiện (detection) và theo dõi (tracking) để đảm bảo số liệu chính xác, hạn chế tình trạng đếm trùng hoặc bỏ sót.

1.2. Tổng quan về bài toán phát hiện đối tượng (Object Detection)

Khái niệm: Phát hiện đối tượng là quá trình xác định vị trí (qua bounding box) và loại của các đối tượng xuất hiện trong ảnh hoặc video. Đây là bước nâng cao so với nhận dạng đơn thuần, vì ngoài phân loại, hệ thống còn phải khoanh vùng chính xác đối tượng.

Phân biệt:

- Classification (Phân loại ảnh): Xác định ảnh chứa đối tượng thuộc loại nào, nhưng không cho biết vị trí.

- Object Detection (Phát hiện đối tượng): Xác định cả loại đối tượng và vị trí của chúng trên ảnh/video.
- Segmentation (Phân vùng ảnh): Xác định ranh giới chi tiết của từng đối tượng đến từng pixel (thường dùng trong y tế, bản đồ, robot).

Thách thức trong Object Detection:

- Vật thể có thể nhiều kích thước khác nhau trong cùng một khung hình.
- Độ sáng, thời tiết, góc quay có thể thay đổi, gây khó khăn cho mô hình.
- Vật thể có thể che khuất nhau hoặc di chuyển nhanh.

1.3. Giới thiệu về mô hình YOLO

YOLO (You Only Look Once) là một thuật toán phát hiện đối tượng trong ảnh nổi bật, thuộc nhóm One-Stage Object Detector, được thiết kế để nhận diện và định vị các vật thể trong thời gian thực.

Khác với các phương pháp truyền thống chia việc phát hiện và phân loại thành hai giai đoạn, YOLO xử lý toàn bộ ảnh đầu vào chỉ trong một lần “nhìn” (one pass) bằng mạng nơ-ron, giúp tăng tốc độ xử lý vượt trội.

YOLOv8s là tên mô hình nhỏ (small) trong bộ mô hình YOLOv8 do Ultralytics phát triển, dùng để phát hiện đối tượng (object detection) được nhóm chúng em sử dụng trong bài báo cáo.

Trong YOLOv8 có nhiều biến thể khác như:

***YOLOv8n → nano (rất nhỏ)**

- Số tham số: ~3.2 triệu
- Kích thước file: ~6 MB
- FLOPs: ~8.7 B
- Tốc độ xử lý (640×640): ~5–7 ms
- FPS tham khảo (GPU RTX 3060): ~150 FPS
- mAP@50-95 (COCO): ~37%

Ưu điểm:

- Nhẹ nhất, phù hợp thiết bị yếu.
- Tốc độ cực nhanh.
- Tối ưu tốt cho nhúng (IoT), Edge AI, Jetson, Raspberry Pi.

Nhược điểm:

- Độ chính xác thấp hơn các phiên bản lớn.
- Không phù hợp bài toán phức tạp.

Dùng khi:

- Hạn chế tài nguyên phần cứng (mobile, microserver)
- Cần tốc độ cực nhanh (≥ 100 FPS)
- Ứng dụng: Nhúng, giám sát đơn giản, thiết bị IoT

Cấu trúc và kiến trúc:

- Backbone: C2f modules thay thế C3 để giảm tham số
- Neck: FPN+PAN giúp trích xuất thông tin ở nhiều cấp độ
- Head: Anchor-free, đơn giản hóa quá trình dự đoán
- Dùng **bộ sinh đặc trưng nhẹ** và số tầng giảm

Ứng dụng:

- Giám sát nhẹ (nhà trẻ, máy bán hàng mini)
- Thiết bị IoT cần tiết kiệm năng lượng
- Phát hiện người qua đường, đếm vật đơn giản

***YOLOv8s → small (nhỏ)**

- Số tham số: ~11.2 triệu
- Kích thước file: ~22 MB
- FLOPs: ~28.3 B
- Tốc độ xử lý: ~8–12 ms
- FPS tham khảo: ~90 FPS
- mAP@50-95: ~44%

Ưu điểm:

- Nhẹ và chính xác hơn YOLOv8n.
- Vẫn chạy tốt trên laptop hoặc GPU tầm trung.
- Dễ huấn luyện và triển khai.

Nhược điểm:

- Chưa đủ tốt cho ứng dụng phức tạp hoặc hình ảnh lộn xộn.
- Có thể thiếu chính xác trong đếm vật nhỏ, nhiều đối tượng.

Dùng khi:

- Cần real-time detection trên PC/laptop/GPU phổ thông
- Dự án cá nhân, startup
- Web app, camera giám sát thời gian thực

Cấu trúc:

- Sử dụng các block C2f mở rộng hơn YOLOv8n
- Cân bằng số tầng và độ sâu mạng
- Hỗ trợ tốt cho multi-scale detection

Ứng dụng:

- App AI chạy thời gian thực: đếm xe, người
- Camera giao thông thông minh mini
- Ứng dụng mobile AI (kết hợp TensorFlow Lite hoặc CoreML)

***YOLOv8m → medium (trung bình)**

- Số tham số: ~25.9 triệu
- Kích thước file: ~47 MB
- FLOPs: ~78.9 B
- Tốc độ xử lý: ~15–20 ms
- FPS tham khảo: ~50 FPS
- mAP@50-95: ~50%

Ưu điểm:

- Cân bằng giữa độ chính xác và tốc độ.
- Phát hiện tốt cả vật nhỏ và phức tạp.
- Ứng dụng được vào hệ thống quy mô vừa.

Nhược điểm:

- Cần GPU tương đối mạnh để huấn luyện (tối thiểu RTX 3060)
- Không lý tưởng cho thiết bị nhúng.

Dùng khi:

- Hệ thống giám sát chất lượng trung bình-khá
- Cân bằng yêu cầu về tốc độ và độ chính xác

- Ứng dụng AI thông minh như: đếm người, nhận diện hàng hóa, phương tiện giao thông

Cấu trúc:

- Backbone có chiều sâu lớn hơn YOLOv8s, giữ nguyên cấu trúc modular
- Neck sử dụng PAN nhiều tầng giúp khôi phục chi tiết
- Head vẫn giữ anchor-free, dễ huấn luyện

Ứng dụng:

- Giám sát an ninh ngoài trời
- Đếm xe/máy trong khung cảnh đông
- Theo dõi đối tượng trong môi trường đông người (tracking)

***YOLOv8l → large (lớn)**

- Số tham số: ~43.7 triệu
- Kích thước file: ~77 MB
- FLOPs: ~165 B
- Tốc độ xử lý: ~25–30 ms
- FPS tham khảo: ~30 FPS
- mAP@50-95: ~52%

Ưu điểm:

- Độ chính xác rất cao.
- Thích hợp xử lý ảnh phức tạp, nhiều vật thể, vật thể nhỏ.
- Ứng dụng tốt trong môi trường sản xuất, giám sát an ninh.

Nhược điểm:

- Cần GPU mạnh (RTX 3080 hoặc tốt hơn)
- Không lý tưởng cho real-time trên thiết bị thông thường.

Dùng khi:

- Yêu cầu độ chính xác cao, độ trễ thấp không quá quan trọng
- Triển khai trong nhà máy, camera giao thông nâng cao, giám sát vùng nguy hiểm

Cấu trúc:

- Mạng sâu, nhiều block C2f

- Số lượng tầng PAN lớn $\Rightarrow$ xử lý vật thể nhỏ tốt hơn
- Phù hợp huấn luyện tập dữ liệu có độ phân giải cao

Ứng dụng:

- Hệ thống giám sát công nghiệp
- Giám sát sản xuất theo dây chuyền
- Giám sát cảng biển, sân bay, biên giới

***YOLOv8x $\rightarrow$ x-large (rất lớn)**

- Số tham số: ~ 68.2 triệu
- Kích thước file: ~ 127 MB
- FLOPs: ~ 258 B
- Tốc độ xử lý: $\sim 35\text{--}50$ ms
- FPS tham khảo: $\sim 15\text{--}20$ FPS
- mAP@50-95: $\sim 53.8\%$

Ưu điểm:

- Chính xác cao nhất trong họ YOLOv8.
- Ứng dụng vào các bài toán phức tạp, dữ liệu nặng.
- Phù hợp huấn luyện mô hình custom trên cloud.

Nhược điểm:

- Tốn tài nguyên, chậm, chỉ phù hợp máy chủ.
- Huấn luyện cần RAM GPU ≥ 16 GB.

Dùng khi:

- Triển khai trên cloud/server GPU mạnh (A100, V100, T4, L4)
- Dự án thương mại quy mô lớn (như AI giám sát thành phố, AI y tế, phân tích video độ phân giải cao)

Cấu trúc:

- Dạng "full depth": số tầng backbone và neck tối đa
- Tích hợp tốt với các framework lớn: ONNX, TensorRT, OpenVINO
- Có thể huấn luyện với batch size lớn trên multi-GPU

Ứng dụng:

- AI trong y tế (nhận diện ảnh CT, MRI)

- AI giám sát giao thông thành phố thông minh
- AI kiểm tra lỗi sản phẩm trong sản xuất (detect defect, lỗi vi mô)

Nó là sự cân bằng giữa tốc độ và độ chính xác — rất lý tưởng để:

- Chạy trên laptop cấu hình trung bình
- Dùng trong hệ thống giám sát giao thông, nhận diện xe
- Tích hợp với các thư viện như **ByteTrack**, **OpenCV**, **Numpy**

* So sánh giữa hai mô hình YOLOv5 và YOLOv8

Tiêu chí	YOLOv5	YOLOv8
Cách sử dụng	Phải import từ repo YOLOv5	Dùng trực tiếp với thư viện Ultralytics
Tích hợp tính năng	Chỉ có object detection	Có detection, segmentation, pose, tracking
Tốc độ & hiệu suất	Tốt	Tốt hơn, nhẹ và chính xác hơn

1.4. Giới thiệu về thư viện ByteTrack

ByteTrack là một thuật toán theo dõi đối tượng (Multi-Object Tracking - MOT), được thiết kế để duy trì ID và theo dõi đường đi của các đối tượng (người, xe, vật thể...) trong video theo thời gian thực.

Khác với nhiều phương pháp truyền thống chỉ theo dõi các đối tượng có độ tin cậy cao (high-confidence detections), ByteTrack thực hiện theo dõi cả những đối tượng có độ tin cậy thấp (low-confidence) – giúp nâng cao độ chính xác, đặc biệt trong các khung hình có vật thể bị che khuất, di chuyển nhanh hoặc bị làm mờ.

Quy trình hoạt động:

1. Nhận kết quả detection từ YOLO (tọa độ box, class, confidence).
2. Gán ID cho các đối tượng dựa trên thuật toán association (kết hợp dựa vào vị trí, tốc độ di chuyển, kích thước box).
3. Cập nhật ID theo thời gian thực, đảm bảo không bị trùng hoặc mất ID.

Ưu điểm:

- Theo dõi ổn định ngay cả khi vật thể biến mất trong vài khung hình.
- Dễ tích hợp với YOLO, OpenCV.

- Phù hợp cho ứng dụng đếm đối tượng di chuyển qua vùng xác định.

1.5. Giới thiệu về thư viện Ultralytics

Ultralytics là một công ty và thư viện mã nguồn mở chuyên phát triển các mô hình trí tuệ nhân tạo, nổi bật nhất là dòng mô hình YOLO (You Only Look Once) – một trong những mô hình hàng đầu trong lĩnh vực phát hiện đối tượng (Object Detection).

Phiên bản YOLOv8 hiện tại là do Ultralytics phát triển từ đầu, không phụ thuộc vào phiên bản YOLOv5 cũ.

Đặc điểm:

- Cài đặt đơn giản chỉ với pip install ultralytics.
- Hỗ trợ huấn luyện mô hình custom chỉ bằng vài dòng lệnh.
- Tích hợp sẵn các mô hình detection, segmentation, pose estimation, tracking.

Ứng dụng:

- Huấn luyện YOLOv8 với dataset tự thu thập.
- Triển khai phát hiện đối tượng trong camera giám sát.
- Kết hợp với ByteTrack để đếm phương tiện giao thông.

CHƯƠNG 2: PHÂN TÍCH VÀ CÀI ĐẶT

2.1. Công cụ sử dụng

Trong quá trình triển khai hệ thống phát hiện và đếm đối tượng, nhóm đã sử dụng các công cụ và thư viện sau:

- Visual Studio Code: Môi trường soạn thảo mã nguồn, hỗ trợ nhiều ngôn ngữ lập trình và tích hợp tốt với Python, có hệ thống mở rộng (extension) phong phú.
- OpenCV: Thư viện xử lý ảnh và video mã nguồn mở, hỗ trợ đọc/ghi video, vẽ bounding box, xử lý ảnh đầu vào trước khi đưa vào mô hình.
- NumPy: Thư viện hỗ trợ tính toán ma trận, xử lý dữ liệu số nhanh chóng, là nền tảng cho nhiều thuật toán thị giác máy tính.
- Mô hình YOLOv8s: Phiên bản nhỏ (small) của YOLOv8, cân bằng giữa tốc độ và độ chính xác, phù hợp chạy thời gian thực trên máy tính cá nhân.
- ByteTrack: Thuật toán theo dõi nhiều đối tượng (Multi-Object Tracking) giúp duy trì ID cho từng đối tượng, tránh đếm trùng hoặc bỏ sót.
- Ultralytics: Thư viện Python chính thức để triển khai, huấn luyện và chạy YOLOv8, giúp việc tích hợp mô hình trở nên đơn giản.

2.2. Phân tích bài toán

Đầu vào:

- Video chứa các đối tượng như người, xe máy, ô tô, xe tải...
- Video có thể được quay từ camera giao thông, camera giám sát hoặc video sẵn có.

Yêu cầu xử lý:

- Phát hiện đối tượng (Object Detection) bằng mô hình YOLOv8.
- Theo dõi và gán ID cho từng đối tượng để tránh đếm trùng.
- Đếm số lượng từng loại đối tượng khi chúng đi qua vùng quan sát đã định nghĩa trước.

Đầu ra:

- Video có gắn bounding box và ID của từng đối tượng.
- Thống kê số lượng từng loại đối tượng đã được đếm.

Ứng dụng thực tế:

- Giám sát giao thông: Thống kê lưu lượng, phát hiện ùn tắc.
- Đếm người vào sự kiện: Giúp quản lý số lượng khách tham dự.
- Thống kê vận chuyển hàng hóa: Đếm số lượng phương tiện ra/vào kho bãi.

2.3. Kiến thức hệ thống tổng quát.

Hệ thống được chia thành 4 bước chính:

1. Đọc video đầu vào:
 - Sử dụng OpenCV để mở và đọc từng khung hình (frame) của video.
 - Có thể thay thế bằng luồng camera trực tiếp nếu muốn chạy thời gian thực.
2. Phát hiện đối tượng bằng YOLOv8:
 - Mô hình YOLOv8s sẽ xử lý từng khung hình để tìm ra vị trí (bounding box), loại đối tượng (class), và độ tin cậy (confidence).
3. Theo dõi đối tượng với ByteTrack:
 - Gán ID cố định cho mỗi đối tượng và duy trì ID này xuyên suốt video.
 - Giúp hệ thống phân biệt các đối tượng khác nhau và tránh đếm nhiều lần cùng một đối tượng.
4. Thống kê / Đếm đối tượng:
 - Khi một đối tượng đi qua vùng xác định, hệ thống ghi nhận và tăng số lượng đếm.
 - Có thể lưu dữ liệu (loại đối tượng, thời gian, vị trí) vào cơ sở dữ liệu để phân tích sau.

2.4. Mô tả mô hình YOLOv8 sử dụng

Tên mô hình: yolov8s.pt (phiên bản small cân bằng giữa tốc độ và độ chính xác)

Đặc điểm chính:

- **Backbone:** Sử dụng kiến trúc **C2f module** – cải tiến từ CSPDarknet, giúp giảm số tham số và tăng hiệu suất xử lý.
- **Neck:** Kết hợp **FPN (Feature Pyramid Network)** và **PAN (Path Aggregation Network)** để trích xuất đặc trưng ở nhiều cấp độ, phát hiện tốt vật thể lớn và nhỏ.
- **Head:** Sử dụng kiến trúc **anchor-free**, giúp dự đoán bounding box trực tiếp, đơn giản hơn so với anchor-based của YOLOv5.

Lý do chọn YOLOv8:

- **Tốc độ nhanh**, có thể chạy gần thời gian thực (real-time).
- Mô hình nhẹ hơn và tối ưu hóa kiến trúc hơn YOLOv5, dễ triển khai trên máy tính cá nhân.
- Tích hợp tốt với thư viện ultralytics.
- Tiết kiệm thời gian phát triển, dễ tinh chỉnh cho dữ liệu.
- YOLOv8 hỗ trợ nhiều tác vụ AI:
 - + Object detection (Nhận diện vật thể).
 - + Instance segmentation (Phân đoạn từng đối tượng).
 - + Pose estimation (Ước lượng tư thế).
 - + Image Classification (Phân loại hình ảnh).

2.5. Mã nguồn chính

```
import cv2
import numpy as np
from ultralytics import solutions
# Kích thước resize video
resize_width = 960
resize_height = 540
# Vùng đếm góc (phù hợp với video gốc lớn)
original_region = [[0, 300], [1500, 300], [1500, 800], [0, 800]]
# Tự động scale lại vùng đếm cho đúng với khung resize
def scale_region_points(region, orig_w, orig_h, new_w, new_h):
    scale_x = new_w / orig_w
    scale_y = new_h / orig_h
    return [[int(x * scale_x), int(y * scale_y)] for x, y in region]
def enhance_dark_blurry(image):
    #Tăng sáng dùng CLAHE trên kênh V
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
    v = clahe.apply(v)
    hsv_enhanced = cv2.merge((h, s, v))
```

```

brightened = cv2.cvtColor(hsv_enhanced, cv2.COLOR_HSV2BGR)
#Làm nét ảnh toàn khung
sharpen_kernel = np.array([[0, -1, 0],
                           [-1, 5, -1],
                           [0, -1, 0]])
sharpened = cv2.filter2D(brightened, -1, sharpen_kernel)
#Làm mờ vùng xa phía trên (giảm vỡ hình khi phóng to)
y1, y2 = 0, int(resize_height * 0.35) # vùng trên 35% ảnh
far_area = sharpened[y1:y2, :]
if far_area.size > 0:
    upscale = cv2.resize(far_area, None, fx=2, fy=2,
interpolation=cv2.INTER_LANCZOS4)
    downscale = cv2.resize(upscale, (far_area.shape[1], far_area.shape[0]),
interpolation=cv2.INTER_LINEAR)
    sharpened[y1:y2, :] = downscale
return sharpened
# Hàm chính: đếm vật thể có đi qua vùng đếm
def count_objects_in_region(video_path, output_video_path, model_path):
    cap = cv2.VideoCapture(video_path)
    assert cap.isOpened(), "không thể mở video"
    orig_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    orig_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    video_writer = cv2.VideoWriter(
        output_video_path,
        cv2.VideoWriter_fourcc(*"mp4v"),
        fps,
        (resize_width, resize_height)
    )
    region_points = scale_region_points(original_region, orig_w, orig_h, resize_width,
resize_height)
    counter = solutions.ObjectCounter(
        model=model_path,
        region=region_points,
        show=False,

```

```

        conf=0.3,
    )
    while cap.isOpened():
        success, frame = cap.read()
        if not success:
            break
        # Resize + xử lý sáng/mờ/mượt vùng xa
        frame = cv2.resize(frame, (resize_width, resize_height))
        enhanced_frame = enhance_dark_blurry(frame)
        # Nhận diện + đếm
        results = counter(enhanced_frame)
        # Ghi video + hiển thị
        video_writer.write(results.plot_im)
        cv2.imshow("kết quả", results.plot_im)
        if cv2.waitKey(1) == ord('q'):
            break
    cap.release()
    video_writer.release()
    cv2.destroyAllWindows()
# Gọi hàm chính
count_objects_in_region(
    video_path="videomo2.mp4",
    output_video_path="output_scaled.avi",
    model_path="yolov8s.pt"
)

```

2.6. Kết quả đầu ra

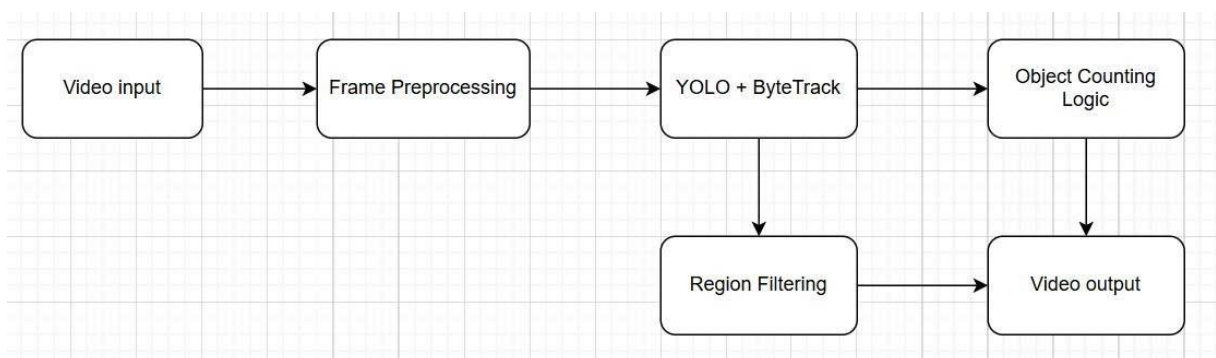
Sau khi thực hiện chương trình, hệ thống đã xử lý video đầu vào và cho ra các kết quả như sau:

- Phát hiện và đếm đối tượng trong vùng quan tâm (ROI):
Các đối tượng như người, xe máy, ô tô được nhận diện chính xác khi đi qua vùng đã xác định trong khung hình.
- Hiển thị khung giới hạn (bounding boxes):
Mỗi đối tượng được hiển thị bằng khung hình chữ nhật, kèm nhãn phân loại và độ tin cậy (confidence score).

- **Đánh số theo dõi (Tracking ID):**
Mỗi đối tượng được gán một ID riêng để theo dõi liên tục giữa các khung hình (dựa vào thuật toán ByteTrack).
- **Giao diện hiển thị kết quả:**
Giao diện hiện bounding box, label và đếm tổng số đối tượng đã đi qua vùng.
Dễ dàng quan sát người, xe, vật thể di chuyển.
- **Video đầu ra:**
Một video mới được tạo ra với tên: *output_scaled.avi*



2.7. Sơ đồ hệ thống



2.8. Nhận xét

Ưu điểm:

- Tốc độ nhanh, có thể xử lý gần thời gian thực (real-time).
- Mô hình nhẹ (yolov8s.pt), dễ cài đặt và triển khai.
- Đếm và theo dõi đối tượng hiệu quả với thuật toán ByteTrack:
- Có thể tùy chỉnh vùng đếm và loại đối tượng dễ dàng.

Nhược điểm:

- Độ chính xác chưa cao với các vật thể nhỏ hoặc bị che khuất.
- Nhạy cảm với điều kiện ánh sáng kém hoặc video chất lượng thấp.
- Phiên bản nhẹ nên độ chính xác thấp hơn các phiên bản lớn hơn như YOLOv8m, YOLOv8l.

CHƯƠNG 3 : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

3.1. Hướng phát triển

Để nâng cao hiệu quả và mở rộng khả năng ứng dụng của hệ thống trong tương lai, nhóm đề xuất một số hướng cải tiến sau:

1. **Nâng cấp mô hình:** Sử dụng các phiên bản YOLO mạnh hơn như YOLOv8m, YOLOv8l hoặc YOLOv11 nhằm cải thiện độ chính xác, đặc biệt với các đối tượng nhỏ, bị che khuất hoặc xuất hiện trong môi trường phức tạp.
2. **Cải thiện tiền xử lý ảnh:** Áp dụng các kỹ thuật image enhancement như cân bằng sáng (histogram equalization), giảm nhiễu (denoising) hoặc tăng cường tương phản để làm rõ vật thể trong điều kiện ánh sáng yếu hoặc video mờ.
3. **Tích hợp giám sát thời gian thực:** Kết nối hệ thống với camera IP hoặc hệ thống CCTV để triển khai trong môi trường giám sát giao thông, quản lý an ninh theo thời gian thực.
4. **Thống kê và báo cáo:** Xây dựng dashboard trực quan hiển thị dữ liệu đếm theo thời gian, biểu đồ lưu lượng đối tượng, và xuất báo cáo hỗ trợ công tác phân tích, quản lý.
5. **Nhận diện chi tiết loại đối tượng:** Kết hợp mô hình phân loại nâng cao để phân biệt chi tiết các loại phương tiện (xe máy, ô tô, xe tải, xe buýt...) hoặc đặc điểm người (nhân viên, khách, người mặc đồng phục...).

3.2. Kết luận

Trong đồ án này, nhóm đã nghiên cứu và xây dựng thành công một ứng dụng sử dụng mô hình YOLOv8 để thực hiện phát hiện và đếm số lượng đối tượng trong video thực tế (cụ thể là người, xe cộ, v.v.).

Thông qua việc tích hợp thư viện Ultralytics, thuật toán theo dõi ByteTrack, cùng các kỹ thuật tiền xử lý ảnh, hệ thống cho thấy khả năng xử lý video nhanh chóng, ổn định và chính xác trong điều kiện thông thường.

Hệ thống cho phép:

- Phát hiện và đánh dấu đối tượng bằng bounding box.
- Theo dõi và gán ID cho từng đối tượng để đếm chính xác khi đi qua vùng quan sát.

- Xuất video kết quả rõ ràng, trực quan, kèm thống kê số lượng.

Các thử nghiệm trên video thực tế cho thấy hệ thống đạt:

- Tốc độ xử lý nhanh, gần thời gian thực.
- Hiệu suất ổn định trong điều kiện ánh sáng và chất lượng video thông thường.

Mặc dù vẫn tồn tại một số hạn chế về độ chính xác khi gặp đối tượng nhỏ, bị che khuất hoặc ánh sáng yếu, hệ thống đã chứng minh tính khả thi và có tiềm năng triển khai trong nhiều lĩnh vực như:

- Giám sát giao thông
- Đếm người trong tòa nhà, sự kiện
- Quản lý an ninh và kiểm soát ra vào

Đây là nền tảng quan trọng để nhóm tiếp tục nghiên cứu, tối ưu và mở rộng các chức năng trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Ultralytics Docs – YOLOv8: <https://docs.ultralytics.com>
- [2] GitHub YOLOv8: <https://github.com/ultralytics/ultralytics>
- [3] ByteTrack Github: <https://github.com/FoundationVision/ByteTrack>
- [4] Tài liệu Elearning Trường Đại Học Văn Lang:
<https://elearning.vlu.edu.vn/course/view.php?id=8219>